Lightweight LiDAR-Based Cooperative Localization Model for Asymmetric Leader-Follower Cooperative Driving Automation System with Asynchronous View Repetition

Yuxin Ding, Chenxi Chen, Xianbiao Hu*

***Abstract*—The Leader-Follower Cooperative Driving Automation (LF-CDA) system, crucial for applications such as truck platooning and off-road vehicle convoys, relies on automation and communication technologies to virtually link multiple vehicles and has become a core focus in the automated vehicle industry. Accurate relative positioning is critical for LF-CDA operations, yet GNSS can be unreliable in challenging environments. Asymmetric architecture is common in many LF-CDA systems, making direct application of localization models either infeasible or both computationally and communication intensive. This manuscript presents a lightweight LiDAR-based cooperative localization model that leverages the unique characteristics of asymmetric LF-CDA systems, specifically the property of "asynchronous view repetition." In this context, the follower vehicle, operating in vehicle-following mode, consistently receives similar visual and spatial information as the leader vehicle, though with a time delay. To capitalize on such system characteristics, an asynchronous view repetition-based graph optimization model is formulated to minimize the positional errors of both leader and follower vehicles. To provide input to and solve the graph optimization model, a lightweight cooperative localization framework with multiple submodules is established, allowing the system to function independently of environmental constraints. A comprehensive set of experiments was conducted in the CARLA simulation environment, using CT-ICP and KISS-ICP as benchmarks, given their strong performance in single-vehicle settings. The results indicate that, under the LF-CDA scenario, our proposed model demonstrates greater suitability by achieving higher localization accuracy while maintaining comparable or even superior computational efficiency.**

***Index Terms*—Leader-Follower Cooperative Driving Automation, Cooperative Localization, Asymmetric Architecture, Cooperative Simultaneous Localization and Mapping (C-SLAM), Graph Optimization, Asynchronous View Repetition.**

I. Introduction

THE Leader-Follower Cooperative Driving Automation (LF-CDA) system has gained significant attention as a practical solution deployable in real-world applications. In such a system, the Leader Vehicle (LV) sets the path while the Follower Vehicles (FVs) mimic its actions, with a focus on path tracking to ensure synchronized task execution. The system relies on automation and communication technologies to virtually connect multiple vehicles, making it a central focus in the automated vehicle industry. Notable examples of LF-CDA system include: (1) *truck platooning*, in which trucks drive in a convoy, communicating and adapting to maintain formation. According to [1], although there are varying levels of automation, the majority of configurations require a driver in the leader vehicle. Truck platooning has been shown to significantly increase fuel efficiency and reduce driver shortages [2], [3]; (2) *off-road vehicle convoys, often used in military applications*. For instance, a truck convoy led by a joint light tactical vehicle is shown to have nine unmanned, cargo-loaded follower vehicles, reducing manpower requirements and saving soldiers' lives when traversing high-threat areas [4]; and (3) the *Autonomous Truck-Mounted Attenuator (ATMA) system*, which was transitioned from military to civilian use for transportation infrastructure maintenance. In such a system, the Leader Vehicle (LV) sets the path while the Follower Vehicles (FVs) mimic its actions, with a focus on path tracking to ensure synchronized task execution. [5], [6].

Although broadly applied across various fields with different configurations, LF-CDA systems commonly employ an asymmetric architecture, where the leader vehicle is responsible for most computing and decision-making tasks and is often equipped with high-performance computing capabilities, while the follower vehicles are generally limited in computing power and decision-making abilities. As the name suggests, LF-CDA systems typically operate within a leader–follower control framework, where each follower vehicle is controlled to track its leader, maintaining a desired relative distance and bearing angle to achieve the prescribed formation geometry based on local sensing capabilities [7]. This centralizes the responsibilities for path planning, decision making, sensor fusion, and communication management in the leader vehicle, effectively making it the "brain" of the platoon. In contrast, the follower vehicle maintains its position relative to the leader and responds to immediate sensor data to handle dynamic changes. It is generally assumed that all vehicles are equipped with sensors for tasks such as object detection and measuring inter-vehicle spacing [8], ensuring that if obstacles—such as a vehicle cutting between the leader and follower or a pedestrian suddenly crossing—arise, the follower vehicle can respond appropriately to avoid collisions. However, since the leader vehicle is responsible for critical tasks within the LF-CDA system, it requires substantial computing power—often needing a high-performance server—whereas the computational demands on follower vehicles are much lower. For instance, it has been noted [9] that follower vehicles are typi-

All authors are with the Department of Civil and Environmental Engineering, Pennsylvania State University (PSU), University Park, PA 16802, USA.

cally lower-cost, computationally limited robots, specifically designed to maintain a desired formation behind a leader vehicle. This suggests that follower vehicles do not require high-performance computing capabilities or a comprehensive perception-localization-decision-making stack, rendering it either infeasible or extremely challenging for them to apply state-of-the-art models for autonomous decision-making.

Accurate relative positioning—i.e., the continuous and stable provision of positional information of the Leader Vehicle and Follower Vehicle, termed the "localization" problem [10] is critical for LF-CDA operations. The most commonly used localization system is the Global Navigation Satellite System (GNSS), which determines the positions of targets using satellite data within a geodetic coordinate framework, thereby establishing the relative positions of system vehicles and maintaining system performance. Based on previous research, using five GNSS devices, the accuracy ranges from 0.11 to 0.17 meters (approximately 4 to 7 inches) [11]. Position errors can be further reduced with Real-Time Kinematic (RTK) correction [12].

However, GNSS coverage and reliability are significantly degraded in areas where signals are obstructed or intermittent, compromising system robustness. Six categories of abnormal GNSS environments have been identified [13]: tree-lined avenues, semi-outdoor areas, urban canyons, under viaducts, shallow indoor spaces, and deep indoor spaces. Unfortunately, these environments are precisely where the LF-CDA system is commonly deployed. Trees often line roadways, vehicles regularly traverse tunnels and underpasses, and skyscrapers in downtown areas create urban canyons. Even on interstate highways with clear skies, GNSS signals can become intermittent, especially in rural areas. Unfortunately, these are precisely the environments in which the LF-CDA system operates. Trees are commonly found along roadways, vehicles often pass through tunnels and underpasses, and urban canyons are present around skyscrapers in downtown areas. Even on interstate highways with open skies, GNSS signals can become intermittent in rural regions. The reliability issues of GNSS, due to insufficient measurements and erroneous data, are also discussed in [14], [15].

To address these challenges, this work focuses on maintaining accurate and robust path tracking within asymmetric LF–CDA systems when GNSS reliability is compromised. We first define the unique characteristics of LF–CDA systems, termed *asynchronous view repetition*, and incorporate this property into a graph optimization formulation that jointly refines the poses of both leader and follower vehicles. Building on this foundation, we develop a lightweight cooperative localization framework with modular subcomponents—including feature-map construction, cross-vehicle registration, temporal synchronization, and iterative optimization—that enables real-time operation under stringent computational and communication constraints. Extensive CARLA-based simulations demonstrate that the proposed approach significantly improves localization accuracy and efficiency compared to state-of-the-art LiDAR-based SLAM baselines, while also clarifying the applicability and boundary conditions under which the method can be reliably deployed.

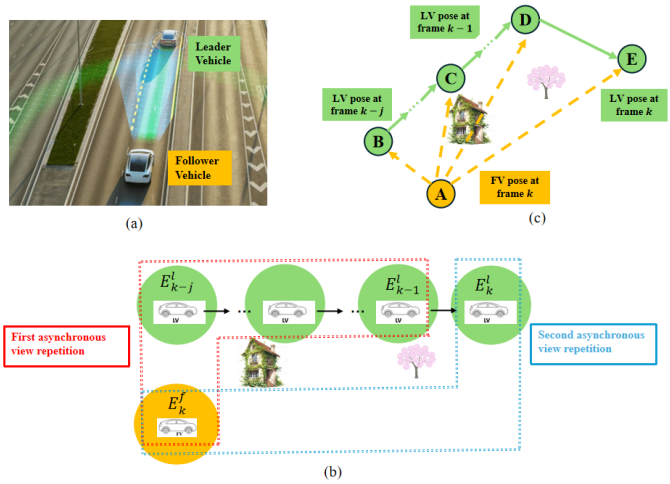


Fig. 1. Illustration of Problem Setting and Asynchronous View Repetition.

In the remainder of this section, we formally define the problem under study and the unique characteristics of LF-CDA, identify the research gap through a literature review, and summarize the contributions of this manuscript.

A. PROBLEM STATEMENT

In this subsection, we formally define the problem under study and introduce the characteristics of LF-CDA systems. For illustration purposes, we refer to Fig. 1, which consists of three subfigures.

Fig. 1(a) illustrates an LF-CDA system comprising two vehicles: a leader vehicle driving in front and a follower vehicle driving behind. Although highway segments are depicted here for illustrative purposes, the proposed LF-CDA system can also be applied to off-road vehicle convoys, where highway segments are absent. For simplicity, a convoy of only two vehicles is shown; however, the system characteristics and subsequent model formulation can readily extend to multiple vehicles.

Fig. 1(b) illustrates the movement of LF-CDA systems and the corresponding odometry pose transitions. As previously discussed, both vehicles are equipped with sensors for object detection, and we use circles to represent the pose trajectory of each vehicle. The odometry poses of the leader vehicle (l) are shown in green color above, while those of the follower vehicle (f) are in yellow color below. The index k represents the LiDAR frame number, and E denotes the odometry pose. For illustration purposes, we assume that at frame k, the leader vehicle is positioned near a tree alongside the roadway, while the follower vehicle is near a house. Throughout this section of the trajectory, both the tree and the house remain within the detection ranges of the leader and follower vehicles, ensuring that both objects are consistently visible and detectable to each vehicle.

Using Fig. 1(b) as a reference, we formally define the unique characteristic of LF-CDA systems, which we term "**asynchronous view repetition**." In an LF-CDA system, the follower vehicle, operating in vehicle-following mode, consistently receives visual and spatial information similar

to that of the leader vehicle, albeit with a time delay. This is exemplified by shared views of objects like houses and trees, as shown in Fig. 1(c). In the following sections, we will develop models that exploit this unique system characteristic to enhance localization accuracy.

Based on the observations in Fig. 1(b), we identify two types of asynchronous view repetitions:

1) **First Asynchronous View Repetition:** The first type of asynchronous view repetition, illustrated by the red dashed box in Fig. 1(b), refers to the view overlaps between the follower vehicle's current view and the leader vehicle's historical view. For illustration purposes, we assume that the leader vehicle passes this location at frame $k-j$. In other words, the historical view of the leader vehicle at frame $k-j$ to $k-1$ (i.e., E_l^{k-j} to E_l^{k-1} that happen between these frames) overlaps with that of the follower vehicle at frame k (i.e., E_k^f at current frame k), allowing both to detect the house and tree.
2) **Second Asynchronous View Repetition:** The second type of asynchronous view repetition, illustrated by the blue dashed box in Fig. 1(b), refers to view overlaps as the follower vehicle moves toward the current location of the leader vehicle. In other words, as the follower vehicle at frame k travels along the designated path toward the leader vehicle's current position, the new LiDAR point clouds it captures at times $k+1, \ldots, k+j$ exhibit an increased degree of view repetition during the transition from pose E_k^f to $E_{k+1}^f, \ldots, E_{k+j}^f$, ultimately approaching the leader vehicle's pose E_k^l.

Overall, the model employs two complementary asynchronous view-repetition constraints: a long-term loop-closure term that matches the follower's current LiDAR scan at time k to the leader's scan at time $k-j$—injecting low-drift geometric context to curb cumulative odometry error—and a zero-lag relative-pose term that directly couples the follower's and leader's poses at time k, enforcing precise instantaneous alignment and compensating any residual drift for improved short-term accuracy.

We address the problem of cooperative vehicle positioning in the absence of GNSS signals. Specifically, when accurate vehicle location from GNSS is unavailable, the challenge is how to utilize sensor data from both the leader vehicle and follower vehicle to accurately determine the positions of all vehicles in the LF-CDA system in real-time. This task is crucial for the system's state estimation and for guiding the follower vehicles to track the LV's trajectory. Additionally, computational efficiency is essential, as the developed system must operate in real-time to ensure smooth and continuous operation. As previously discussed, asymmetric architecture is common in many LF-CDA systems, where the follower vehicles do not require high-performance computing capabilities or a full perception-localization-decision-making stack. Thus, requiring the follower vehicle to directly apply state-of-the-art models for independent decision-making is either infeasible or highly challenging. The mathematical notations used in this paper are summarized in Table I.

TABLE I
NOTATION LIST

Notation	Definition
i	This is used to denote the points within the point cloud.
k	This is used to denote the LiDAR frame number, i.e., time stamp.
l, f	The subscript l represents the leader vehicle, and the subscript f denotes the follower vehicle.
fs, fl	The subscript fs and fl are used to indicate the distinction between the current frame of the following vehicle and the historical frames of the leader vehicle in the graph optimization module.
$\boldsymbol{\omega}_k$	Angular velocity of kth point cloud.
$\boldsymbol{v}_k$	The velocity of kth point cloud.
$\boldsymbol{a}_{n-1}^{n,IMU}, \boldsymbol{a}_{n-1}^{n,LiDAR}$	The change in angle between two consecutive sensor points.
$\boldsymbol{p}_k^i, \tilde{\boldsymbol{p}}_k^i$	The ith point in k^{th} frame before and after removing distortion, $\boldsymbol{p}_k^i, \tilde{\boldsymbol{p}}_k^i \in \mathbb{R}^3$.
$\boldsymbol{R}_{LiDAR}^{IMU}$	The rotation from LiDAR to IMU odometer, $\boldsymbol{R}_{LiDAR}^{IMU} \in SO(3)$.
$\boldsymbol{R}_{k-1}^{k,LiDAR}$	The rotation between $k-1^{th}$ frame and k^{th} LiDAR cloud estimated by IMU, $\boldsymbol{R}_{k-1}^{k,LiDAR} \in SO(3)$.
$\boldsymbol{t}_{k-1}^k$	The translation between $k-1^{th}$ frame and k^{th} LiDAR cloud estimated by IMU, $\boldsymbol{t}_{k-1}^k \in T(3)$.
$\boldsymbol{T}_{k-1}^{k,f}$	The transformation between $k-1^{th}$ frame and k^{th} LiDAR cloud estimated by IMU, $\boldsymbol{T}_{k-1}^{k,f} \in SE(3)$.
$\boldsymbol{E}_k^l, \boldsymbol{E}_k^f$	The pose of the leader's and follower's odometry corresponds to k^{th} frame point cloud data output from the proposed model, $\boldsymbol{E}_k^L, \boldsymbol{E}_k^f \in SE(3)$.
$\boldsymbol{L}_k^l, \boldsymbol{L}_k^f$	The ground truth poses of the leader's and follower's odometry corresponds to k^{th} frame point cloud data, $\boldsymbol{L}_k^L, \boldsymbol{L}_k^f \in SE(3)$.
$\boldsymbol{T}_k^{fs}$	The transformation matrix that converts the pose of the follower vehicle associated with the kth frame point cloud to the nearest historical pose of the leader vehicle, $\boldsymbol{T}_k^{fs} \in SE(3)$.
$\boldsymbol{T}_k^{fl}$	The transformation matrix that converts the current pose of the follower vehicle to the current pose of the leader vehicle (k^{th} frame), $\boldsymbol{T}_k^{fl} \in SE(3)$.
$\boldsymbol{M}_k$	The feature map corresponding to LV k^{th} frame.
$\boldsymbol{P}_k, \tilde{\boldsymbol{P}}_k$	The kth frame LiDAR cloud before and after removing distortion.

The inputs to this system are as follows: (1) the raw LiDAR point clouds from both the leader vehicle and the follower vehicles, denoted as $\boldsymbol{P}^l$ and $\boldsymbol{P}^f$, respectively, where l refers to the leader vehicle and f refers to the follower vehicle, and (2) raw IMU data, which includes angular velocity $\boldsymbol{\omega}$ and acceleration $\boldsymbol{a}$. Research has consistently shown that one of the primary advantages of using IMUs is their ability to provide accurate acceleration and orientation information even in environments where GNSS signals may be weak or unavailable [16]. This is because IMUs rely on internal sensors rather than external signals to determine a vehicle's location and orientation. Furthermore, IMUs can enhance the accuracy and precision of data collected by providing real-time corrections for errors caused by factors such as device tilt or

movement.

The system outputs the poses of both vehicles, which are denoted as $\mathbf{E}^f$ and $\mathbf{E}^l$, for the follower and leader, respectively. Let $\mathbf{L}^f$ and $\mathbf{L}^l$ represent the ground truth poses of the follower's and leader's odometry, respectively. We can compute the position error as $\boldsymbol{e}_k^\pi = d(\mathbf{L}_k^\pi, \mathbf{E}_k^\pi)$, where d denotes a distance metric that accounts for both translation and rotation differences between the estimated pose $\mathbf{E}_k^\pi$ and the ground truth pose $\mathbf{L}_k^\pi$. Here, $\pi \in \{f, l\}$ denotes the follower and leader vehicles, respectively.

Finally, we convert the cooperative vehicle positioning problem into a graph, as shown in Fig. 1(c). In the LF-CDA system, our objective is to minimize the positional error of both the leader and follower vehicles. To achieve this, we define two types of positioning errors as the optimization targets in our graph-based framework.

1) **Positioning error of the leader vehicle (green lines in Fig. 1(c)**: This error occurs during the movement of the leader vehicle from a previous point cloud, denoted as $k-1$, to a subsequent point cloud, denoted as k, and represents the state accuracy of the leader vehicle. In cases of continuous GNSS signal loss, the current position of the leader vehicle, $\boldsymbol{E}_k^l$, is inferred using the previously determined position, $\boldsymbol{E}_{k-1}^l$, along with estimates of speed, heading (or direction/course), and elapsed time. This process, known as dead reckoning, can lead to an accumulation of error over time. We represent this positioning error as $\boldsymbol{e}_k^l$, which depends on both $\boldsymbol{E}_{k-1}^l$ and $\boldsymbol{E}_k^l$. Thus, the error can be expressed as $\boldsymbol{e}_k^l(\boldsymbol{E}_{k-1}^l, \boldsymbol{E}_k^l)$. This type of positioning error corresponds to the green lines in Fig. 1(c).
2) **Positioning error between the leader and follower vehicles (yellow dashed line in Fig. 1(c)**: This error reflects the system's inability to maintain a consistent spacing gap between the leader and follower vehicles, which is crucial for the safety of LF-CDA systems. As shown in Fig. 1, if the vehicles are positioned accurately, two types of view repetition should be observed, and thus the positioning error should be zero. To generalize, we denote these positioning errors as $\boldsymbol{e}_k^{fs}(\boldsymbol{E}_k^f, \boldsymbol{E}_{k-j}^l)$ and $\boldsymbol{e}_k^{fl}(\boldsymbol{E}_k^f, \boldsymbol{E}_k^l)$, respectively. These correspond to the yellow dashed line in Fig. 1(c).

The objective of this manuscript is to minimize these two types of positioning errors. As discussed earlier, type one positioning error corresponds to the green lines in 1(c), while type two positioning error corresponds to the yellow dashed line in 1(c). Consequently, the problem transitions into a graph-based optimization problem with constraints. The detailed model formulation is presented in the next section.

B. RELATED WORKS ON COOPERATIVE LOCALIZATION

In this subsection, we review prior research related to cooperative localization in multi-agent systems, which can be categorized into two main approaches: agent-to-infrastructure (A2I) and agent-to-agent (A2A).

The core principle of A2I-based methods is to leverage the exchange of relative positional information between vehicles and strategically placed infrastructure. These methods rely on the deployment of communication infrastructure, such as base stations [17], positioned along roads and in designated areas to facilitate continuous communication and data exchange with passing vehicles. A particularly popular approach in A2I-based methods is LTE signal-based navigation [18], [19], where localization is achieved through the interaction between vehicles and cell towers emitting LTE signals. Additionally, the model developed in [20] involves vehicles communicating with the infrastructure when they enter its operational range. This model determines the relative positions of the vehicles and the infrastructure by constructing a factor graph, which uses the vehicles' topological arrangement to compute each vehicle's relative pose, enabling cooperative localization. Kim et al. [21] utilize infrastructure as a repository for high-definition (HD) maps, which are made available to nearby vehicle groups for localization. While A2I-based methods offer high precision, they come with notable drawbacks, primarily the high cost of infrastructure deployment. This expense limits the possibility of achieving full coverage [17], making A2I-based methods constrained by their operational environments. To overcome the limitations of infrastructure deployment, our proposed localization method operates independently of any base station support; therefore, A2I-based methods are not included in our approach.

A2A-based methods utilize the onboard systems of vehicles to exchange data and determine their relative positions using Cooperative Simultaneous Localization and Mapping (C-SLAM) technology, enabling cooperative localization within multi-vehicle groups. Based on the server's location, these methods can be divided into two categories: centralized and decentralized [22].

Centralized A2A methods collect and process data from all connected vehicles within a group [23], estimating the positions of agents based on the shared information. This integrated approach makes the performance of centralized systems heavily dependent on the capabilities of a central processor. To address this limitation, Nagatani et al. [24] propose a method where each agent independently constructs a submap, which is later fused offline to create a global map based on inter-robot constraints. However, this process is conducted offline, rather than in real time. Similarly, Michael et al. [25] have agents construct local submaps individually, merging them offline, with backend optimization and localization tasks performed using the Iterative Closest Point (ICP) method. However, both methods lack real-time capabilities. To overcome this, Riazuelo et al. [26] propose a centralized mapping system where intensive map optimization and storage tasks are offloaded to a cloud server, while inter-vehicle relative poses are estimated using a lightweight camera. In contrast, rather than using a cloud server, the multi-vehicle group in [27] employs one high-performance processor on a single agent to handle backend optimization and localization. Fang et al. [28] develop a cooperative localization system for autonomous vehicles that uses shared information from neighboring vehicles and manages various types of errors in state estimation. This system employs point cloud registration and the Adaptive Cubature Split Covariance Intersection Filter (ACSCIF) algorithm to

manage nonlinearity, dynamic noise, and source correlations. In summary, centralized methods rely on a high-performance processing center, located either on a cloud server or within a specific agent, which aggregates data from all agents and performs unified optimization. This approach aligns well with the asymmetric architecture of LF-CDA systems, where the leader vehicle is responsible for most computing and decision-making tasks, while the follower vehicles do not require high-performance computing. However, this framework may face two challenges: first, transmitting data from each vehicle to a central processor places a significant load on communication channels; second, the central processor must handle a large volume of information simultaneously, demanding substantial computational resources.

Decentralized A2A methods operate without a central server, distributing computational tasks across individual vehicle nodes. This approach allows each vehicle to process data locally, enhancing system resilience and scalability [23]. The primary distinction between decentralized and centralized systems lies in the estimation of relative poses between vehicles. Instead of estimating all agents' poses on a single server, each agent independently estimates its pose using a shared global map within its local server [29]. A common decentralized method for inter-robot relative pose estimation involves directly detecting one robot's position relative to another [30]. In one study [31], researchers used time-of-flight distance measurements to compute relative poses between underwater vehicles. Another study [32] utilized image recognition algorithms and spatial position transformations to determine relative poses, with aerial imagery filtering out noise. Similarly, Kim et al. [33] equipped each robot with special markers to enhance vision-based relative pose estimation, which served as constraints in pose graph optimization. However, these methods require line-of-sight visibility between robots, limiting their applicability in diverse environments. To overcome this challenge, an alternative approach involves aligning observations made by different agents of the same scene, a process known as loop closure. This module improves map accuracy by identifying and connecting overlapping observations, allowing for relative pose estimation by comparing shared environmental perceptions and the global map [34], [35]. Notable examples include collaborative structure-from-motion (CSfM) [36], collaborative visual SLAM (CoSLAM) [37], and DDF-SAM [38], [39], all of which use shared information to estimate relative poses between agents. While such loop closure-based approach reduces the need for line-of-sight visibility, it demands substantial communication bandwidth due to the large volume of shared data between robots. As the number of robots increases, the volume of exchanged data grows quadratically, which degrades real-time performance. To address this, some researchers modified the communication protocol from "one-to-all" to "one-to-one," reducing bandwidth usage[29]. Others focused on compressing shared data. For instance, Wu et al. [40] transformed the global map into a heat map, significantly reducing its size. Similarly, Lynen et al. [41] compressed the global map using feature descriptors, lessening the bandwidth burden in multi-agent systems. However, in both cases, the feature map construction was performed offline, imposing limitations on real-time multi-agent system operations.

In summary, while extensive research has been conducted on cooperative localization for multi-agent systems, directly applying these methodologies to LF-CDA systems has proven infeasible or highly challenging due to their high computational demands and significant communication burdens. This is especially the case due to the asymmetric architecture of LF-CDA systems, where the follower vehicles are not equipped with high-performance computing capabilities or a full perception-localization-decision-making stack. To address these challenges, a centralized A2A framework is adopted, and a lightweight cooperative localization model that reduces computational overhead and minimizes communication requirements is proposed. The proposed model is LiDAR-based and designed to function in challenging environments where GNSS signals are weak or unavailable.

As discussed, the goal of this study is to maintain accurate and robust path-tracking within LF-CDA systems when GNSS reliability is compromised. By leveraging the repetitive view feature inherent in LF-CDA systems, we develop a lightweight, real-time cooperative localization model to ensure continuous and safe system performance during GNSS signal loss.

The contributions of this study are threefold:

1) **Graph Optimization-based Problem Formulation that Leverages asymmetric LF-CDA Characteristics**: We formally define the unique characteristics of asymmetric LF-CDA operation, termed "asynchronous view repetition," and formulate the minimization of two types of positioning errors within the LF-CDA system as a graph optimization model. This new formulation allows us to maintain high localization accuracy and robustness, while saving computational resources and enhancing real-time performance.
2) **Lightweight Cooperative Localization Framework**: To provide input to and solve the graph optimization model, we present a lightweight cooperative localization framework for asymmetric LF-CDA systems. This framework includes IMU-assisted point cloud feature extraction, robust feature map construction, feature map-based follower vehicle pose estimation, and decoupled time synchronization modules. These modules enable the system to function independently of environmental constraints while ensuring the orderly transmission of system data streams and maintaining real-time performance.
3) **Satisfactory System Performance**: We implement the developed modeling framework and demonstrate its compatibility with the asymmetric architecture of LF-CDA systems. Numerical analysis indicates that, under the asymmetric LF-CDA setting, the proposed model demonstrates superior suitability for multi-vehicle localization compared to two state-of-the-art algorithms, CT-ICP and KISS-ICP, which were originally designed for single-vehicle scenarios. Additionally, the model achieves comparable or higher computational efficiency.

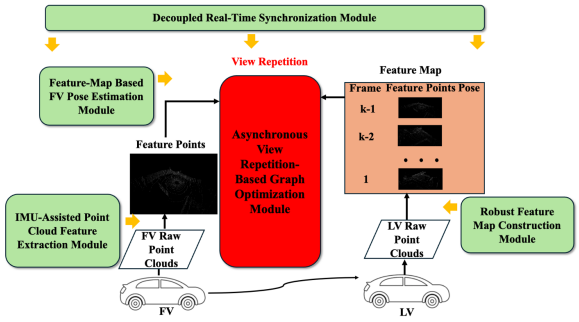


Fig. 2. Illustration of Developed Modeling Framework.

II. METHODOLOGY

This section presents the proposed lightweight cooperative localization model for the LF-CDA system, designed to enable follower vehicles to track trajectories without relying on GNSS data. First, we formulate the cooperative localization problem for the LF-CDA system as a graph-based optimization problem, defining the objective function and constraints. Next, a series of submodules is introduced to provide inputs to the formulated graph optimization problem. Finally, an iterative nonlinear optimization algorithm is developed to deliver efficient and accurate solutions.

Fig. 2 illustrates the developed modeling framework, centered around the asynchronous view repetition-based graph optimization model, which receives and processes input data from both the leader and follower vehicles. The model works as follow: (1) First, both the leader vehicle and follower vehicle are equipped with LiDAR sensors to collect raw point cloud data representing their surrounding environments. (2) An *IMU-assisted lightweight point cloud feature extraction module* then extracts key feature points from this raw data. (3) On the LV side, a *robust feature map construction module* processes the LV's feature point cloud, localizes the LV, and constructs a feature map index. (4) The FV's feature points are transmitted to the LV via V2V communication, where they are combined with the feature map to estimate the FV's pose using *feature map-based FV pose estimation module*. (5) The feature map and estimated pose of the FV are then input into the *asynchronous view repetition-based graph optimization model* to reduce cumulative errors for both vehicles, significantly enhancing localization accuracy. Finally, (6) a *decoupled real-time synchronization module* ensures the system's synchronization and real-time performance. The proposed framework, along with the five main modules discussed above, is summarized in Table II below.

TABLE II
PSEUDOCODE OF THE PROPOSED MODELING FRAMEWORK

Model: *Lightweight LiDAR-Based Cooperative Localization Model for Asymmetric LF-CDA System in GNSS-Denied Environments*

Initialization: Initialize the system by activating the LiDAR sensors of the vehicles and determining the origin of the world coordinate system.

Input: LiDAR point cloud data $\boldsymbol{P}_k^{\pi}$, where $\pi \in \{f, l\}$, from both the leader and follower vehicles in the LF-CDA system at frame k. Additional, the corresponding IMU data, denoted as $\boldsymbol{\omega}_k, \boldsymbol{v}_k$, respectively.

Begin:

1) For both the leader and follower vehicles, use the *IMU-assisted lightweight point cloud feature extraction module* to process $\boldsymbol{P}_k^{\pi}$ and output $\tilde{\boldsymbol{P}}_k^{\pi}$, the feature points after removing distortions.
2) Transmit the FV's feature points to the LV via V2V communication.
3) On the leader vehicle:
   a) Take $\tilde{\boldsymbol{P}}_k^{l}$ as input, and the *robust feature map construction module* will estimate the pose of the leader $\boldsymbol{E}_k^{l}$ vehicle and update the feature map $\boldsymbol{M}_k = \boldsymbol{M}_{k-1} + (\tilde{\boldsymbol{P}}_k^{l}, \boldsymbol{E}_k^{l})$.
   b) The *feature map-based FV pose estimation module* will utilize $\tilde{\boldsymbol{P}}_k^{f}$ from the follower vehicle, along with the feature map $\boldsymbol{M}_k$, to determine the pose of the follower vehicle $\boldsymbol{E}_k^{f}$.
   c) The *asynchronous view repetition-based graph optimization model* will use some poses $\boldsymbol{E} = (\boldsymbol{E}_{k-j}^{l}, \ldots, \boldsymbol{E}_k^{l})$ from $\boldsymbol{M}_k$ and $\boldsymbol{E}_k^{f}$ as inputs to calculate the accurate poses of both the leader and follower vehicles, and the output is the updated poses $\boldsymbol{E}_{\text{out}} = (\boldsymbol{E}_{k-j}^{l}, \ldots, \boldsymbol{E}_k^{l})$ and the updated follower vehicle's pose $\boldsymbol{E}_k^{f}$.
   d) The poses in $\boldsymbol{M}_k$ for frames $k-j$ to k will be updated by $\boldsymbol{E}_{\text{out}}$ from the *asynchronous view repetition-based graph optimization model*.
4) The *decoupled real-time synchronization module* oversees the entire process, ensuring the system operates in real-time.

Output: The poses of both the leader and follower vehicles $\boldsymbol{E}_k^{l}, \boldsymbol{E}_k^{f}$, and the updated feature map $\boldsymbol{M}_k$.

End.

A. ASYNCHRONOUS VIEW REPETITION-BASED GRAPH OPTIMIZATION MODULE FORMULATION

Central to the proposed modeling framework is the formulation of the asynchronous view repetition-based graph optimization module, which leverages the unique characteristics of the LF-CDA system. A complete graph optimization module involves both constructing the graph and solving the objective function. Traditional graph optimization algorithms require substantial computational resources to establish the graph, primarily due to their global search strategy for identifying loop closure points and matching point clouds to create loop closure edges. However, for LF-CDA systems, this global search strategy is inefficient and often wasteful of computational resources. Therefore, in this module, we develop a graph optimization approach tailored to LF-CDA systems. This approach abandons the reliance on computationally intensive global search strategies, accelerating point cloud matching by fully exploiting the high redundancy in views among vehicles. This not only improves operational speed but also eliminates cumulative errors, significantly enhancing the precision of front-end localization. The mainstream graph optimization frameworks include g2o [42], Ceres [43], GTSAM [44], and SE-Sync [45]. Based on an evaluation of operational speed, flexibility, and prior research, we adapt the GTSAM framework as the basis for our graph optimization module.

We first extend the simplified illustration from Fig. 1(c) to Fig. 3, where we identify key points for loop closure. As in Fig. 1(c), the leader vehicle is represented in green and the follower vehicle in yellow. Each vehicle has a sequence of poses at different frames. At a particular frame k, without

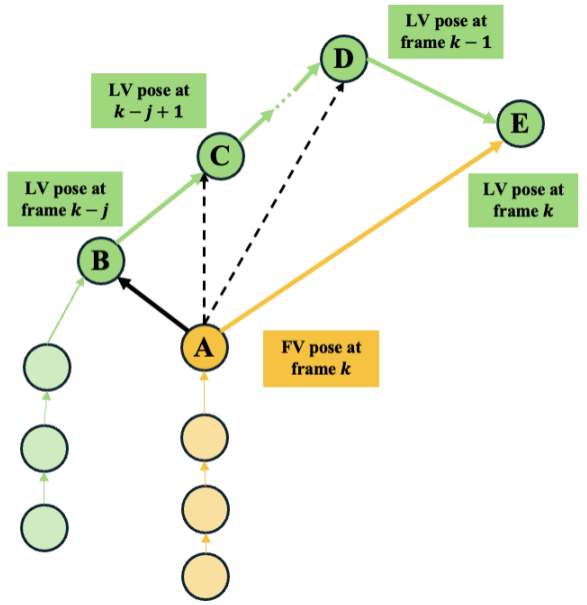


Fig. 3. Graphical representation of pose relationships and loop closure identification.

loss of generality, we denote the follower vehicle's pose as node A and the leader vehicle's pose as node E.

A loop closure in the LF-CDA system is defined as follows. First, we search the prior poses of the leader vehicle and identify the pose from the feature map that is closest to the current pose of the follower vehicle. We denote this pose as Node B, with a LiDAR frame ID of $k-j$. A loop closure in the LF-CDA system is then constructed using three chains: (1) connecting the poses of the leader vehicle from LiDAR frame $k-j$ to frame k; (2) connecting the follower vehicle's pose at frame k to the leader vehicle's pose at frame $k-j$; and (3) connecting the follower vehicle's pose at frame k to the leader vehicle's pose at frame k. In Fig. 3, these correspond to edges B–C–...–D–E, A–B, and A–E. Together, these sets of edges form a closed loop.

To establish the graph optimization equations, we rely on key transformation relationships, represented by bold lines with arrows. The bold green lines denote direct transformations between consecutive poses of the leader vehicle, which are computed through the *robust feature map construction module* (to be discussed later). The bold black and orange lines represent the relative poses from the leader to the follower vehicle, estimated by the *feature-map-based FV pose estimation module* (to be discussed later). There are two such relative poses: the shorter one (black) symbolizes the connection between the current pose (at the kth frame) of the follower vehicle and the closest historical pose (at the $k-j^{\text{th}}$ frame) of the LV in the feature map, while the longer one (orange) represents the current relative pose (at the kth frame) from the LV to the FV. In this graph, the pose of the FV serves as a bridge, linking the historical poses of the LV, effectively mitigating cumulative errors and fully exploiting the operational characteristics of LF-CDA systems. Additionally, based on the headway distance between the LV and FV, more edges may be established during graph optimization, as illustrated by the black dashed lines in Fig. 3.

Based on the nodes and edges depicted in Fig. 3, the objective function can be formulated as follows:

$$\begin{aligned}\min F(\boldsymbol{E}) = \sum_{i=k-j}^{k} & \boldsymbol{e}_i^l(\boldsymbol{E}_i^l, \boldsymbol{E}_{i-1}^l)^\top \Delta_i^l \boldsymbol{e}_i^l(\boldsymbol{E}_i^l, \boldsymbol{E}_{i-1}^l) \\ & + \alpha \boldsymbol{e}_k^{fs}(\boldsymbol{E}_k^f, \boldsymbol{E}_{k-j}^l)^\top \boldsymbol{e}_k^{fs}(\boldsymbol{E}_k^f, \boldsymbol{E}_{k-j}^l) \\ & + \beta \boldsymbol{e}_k^{fl}(\boldsymbol{E}_k^f, \boldsymbol{E}_k^l)^\top \boldsymbol{e}_k^{fl}(\boldsymbol{E}_k^f, \boldsymbol{E}_k^l) \end{aligned} \quad (1)$$

This equation consists of three components corresponding to the green, black, and orange line segments shown in Fig. 3. The green line segments represent constraints on the LV's trajectory poses, while the black and orange segments represent constraints on the relative poses between the LV and FV.

Here, Δ_l^i is a weight factor that indicates the importance assigned to each pose of the leader vehicle. Due to the cumulative error impact, the current pose of the LV is most affected by these errors and is thus assigned the least importance. In contrast, the pose farthest from the current LV pose—closest to the FV's current pose—is least affected by cumulative errors and is given the highest importance. Δ_l^i for intermediate nodes is linearly interpolated.

Similarly, σ and β represent the importance of the error terms between the follower and leader vehicles. In this model, we set σ equal to Δ_l^{k-j} and β equal to Δ_k^l. For the error terms:

- The error $\boldsymbol{e}_i^l$ represents the pose error between two consecutive point clouds, at the ith and $i+1^{\text{th}}$ frames of the leader vehicle. This can be computed using the rotation matrix $\boldsymbol{R}_{i-1}$ and translation matrix $\boldsymbol{t}_{i-1}$, which are derived from the *feature map construction module* (to be discussed later):

$$\boldsymbol{e}_i^l(\boldsymbol{E}_i^l, \boldsymbol{E}_{i-1}^l) = \boldsymbol{E}_i^l - \begin{bmatrix} \boldsymbol{R}_{i-1} & \boldsymbol{t}_{i-1} \\ \boldsymbol{0} & 1 \end{bmatrix} * \boldsymbol{E}_{i-1}^l \quad (2)$$

- The errors $\boldsymbol{e}_k^{fs}$ and $\boldsymbol{e}_k^{fl}$ represent the pose errors between the follower vehicle and the leader vehicle under the constraints of their relative poses, which are derived from the *feature-map-based FV pose estimation module* (to be discussed later). These errors are computed as follows:

$$\boldsymbol{e}_k^{fs}\left(\boldsymbol{E}_k^f, \boldsymbol{E}_{k-j}^l\right) = \boldsymbol{T}_k^{fs} * \boldsymbol{E}_k^f - \boldsymbol{E}_{k-j}^l \quad (3)$$

$$\boldsymbol{e}_k^{fl}\left(\boldsymbol{E}_k^f, \boldsymbol{E}_k^l\right) = \boldsymbol{T}_k^{fl} * \boldsymbol{E}_k^f - \boldsymbol{E}_k^l \quad (4)$$

Here, $\boldsymbol{T}_k^{fs}$ and $\boldsymbol{T}_k^{fl}$ correspond to the respective relative pose transformations calculated by the *feature-map-based FV pose estimation module*.

As can be seen above, as the core component of the entire model, this module takes two inputs: the feature map $\boldsymbol{M}_k$ at step k, output by the *robust feature map construction module*, and the pose of the FV, $\boldsymbol{E}_k^f$ at time step k, generated by the *feature-map-based FV pose estimation module*. Next, we will delve into the details of these modules.

B. ROBUST FEATURE MAP CONSTRUCTION

In this module, we create a container called the feature map, which stores feature points extracted from the leader vehicle's historical point clouds, $\boldsymbol{P}_k^l$, and their corresponding poses, $\boldsymbol{E}_k^l$. The primary goal of this module is to establish relative pose relationships between the LV and the FV, thereby achieving precise localization.

The construction of the feature map is divided into two steps: first, estimating the pose of the current point cloud from the FV, and second, extracting feature points from this point cloud frame. These feature points, along with their corresponding poses, are then packaged and stored in the feature map container. Although SLAM technologies have been extensively studied, most existing systems rely on specific assumptions about robot movement, such as those used in LOAM [46], and environmental structures, like CT-ICP [47], to ensure precise and stable outcomes. However, complex driving environments often invalidate these assumptions, making them impractical. To address more challenging driving conditions and enhance the robustness of our model, the KISS-ICP algorithm [48] is adapted to work with the LF-CDA system for our specific problem, which is used to estimate the pose of the LV corresponding to k^th frame, $\boldsymbol{E}_{\text{pred},k}^l$. After obtaining the pose of the LV's LiDAR odometry, the *IMU-assisted lightweight point cloud feature extraction module* is used to extract features from the point cloud of that frame, resulting in $\tilde{\boldsymbol{P}}_k^l$. The extracted feature point cloud $\tilde{\boldsymbol{P}}_k^l$, along with its corresponding pose $\boldsymbol{E}_k^l$, updates the feature map as:

$$\boldsymbol{M}_k = \boldsymbol{M}_{k-1} + \left(\tilde{\boldsymbol{P}}_k^l, \boldsymbol{E}_k^l\right) \tag{5}$$

C. FEATURE MAP-BASED FOLLOWER VEHICLE POSE ESTIMATION

The purpose of this module is to estimate the pose of the follower vehicle (FV), $\boldsymbol{E}_k^f$, within the feature map $\boldsymbol{M}_k$, using point cloud registration to achieve this estimation. Point cloud registration algorithms can generally be categorized into two types: those based on Iterative Closest Point (ICP) [49] and those based on Normal Distributions Transform (NDT) [50]. Since ICP-based methods are highly sensitive to initial pose estimates and can be time-consuming when processing large point clouds, this module adopts NDT-based point cloud registration. Given the typically large size of point clouds generated by vehicular LiDAR, directly applying the NDT algorithm can demand substantial computational resources. To mitigate this, point cloud registration in this module is conducted in two stages. First, the FV's pose is roughly estimated using IMU sensor data. Based on this estimated pose, some prior relative poses are calculated and used as the initial input for the NDT algorithm, allowing for a more precise relative pose estimation.

First, we use the pose of the follower vehicle from the $(k-1)^{\text{th}}$ frame to estimate its pose in the kth frame. To achieve this, we need the rotation matrix and translation vector between the two frames. The rotation matrix $\boldsymbol{R}_{k-1}^{k,Lidar}$ will be discussed in later sections; here we will calculate the translation vector. Given that the vehicle's state does not undergo significant changes over short time intervals, we approximate the current translation using the translation from the previous frame, resulting in:

$$\boldsymbol{t}_{k-1}^k = \boldsymbol{U}_{k-1}^f - \boldsymbol{U}_{k-2}^f \tag{6}$$

Where the $\boldsymbol{U}^f$ represents the position of the FV. The transformation matrix between the $(k-1)^{\text{th}}$ frame and k^{th} frame can be calculated as:

$$\boldsymbol{T}_{k-1}^{k,f} = \begin{bmatrix} \boldsymbol{R}_{k-1}^{k,Lidar} & \boldsymbol{t}_{k-1}^k \\ \boldsymbol{0} & 1 \end{bmatrix} \tag{7}$$

Using this transformation matrix, the pose of the FV in the kth frame can be roughly estimated as:

$$\hat{\boldsymbol{E}}_k^f = \boldsymbol{T}_{k-1}^{k,f} * \boldsymbol{E}_{k-1}^f \tag{8}$$

To implement the NDT algorithm for estimating the pose of the follower vehicle, it is necessary to define the source and target point clouds. The source point cloud is the current point cloud from the FV, and the target point cloud is determined as follows:

1) In the feature map container $\boldsymbol{M}_k$, search for the oth frame feature point cloud $\tilde{\boldsymbol{P}}_o^l$ of the leader vehicle that is closest to the estimated pose $\hat{\boldsymbol{E}}_k^f$. The pose of the LV's o^{th} frame feature point cloud is $\boldsymbol{E}_o^l$.
2) Integrate the feature point clouds in $\boldsymbol{M}_k$ from frames $(o-n)^{\text{th}}$ to $(o+n)^{\text{th}}$ to create a local map, which will serve as the target point cloud.

Next, $\hat{\boldsymbol{E}}_k^f$ is used to estimate the initial transformation matrix between the source and target point clouds, using the following equation:

$$\boldsymbol{T}_{\text{initial}} = \boldsymbol{E}_o^l * \left(\hat{\boldsymbol{E}}_k^f\right)^{-1} \tag{9}$$

This initial estimate is then used as the prior value for the NDT algorithm.

To implement the NDT algorithm using the target point cloud, the source point cloud, and the initial transformation estimate $\boldsymbol{T}_{\text{initial}}$, we first divide the target point cloud into individual voxels. For each voxel, the mean μ and variance σ of the coordinates of the points within the voxel are calculated. The probability function corresponding to each voxel is given by:

$$p(\boldsymbol{T}, \boldsymbol{P}_{\text{source}}) = \frac{1}{(2\pi)^{\frac{3}{2}}\sqrt{|\sigma|}} * \exp\left(-\frac{(\boldsymbol{P}_{\text{source}}(\boldsymbol{T}) - \mu)^\top \sigma^{-1} (\boldsymbol{P}_{\text{source}}(\boldsymbol{T}) - \mu)}{2}\right) \tag{10}$$

Here, μ and σ are the mean and variance values of the voxel, calculated based on the three-dimensional coordinates of the points within each voxel. $\boldsymbol{P}_{\text{source}}(\boldsymbol{T})$ represents the points of the source cloud after applying the transformation corresponding to $\boldsymbol{T}$. Next, we determine the optimized transformation $\boldsymbol{T}_k^{T2S}$ between the source and target point clouds, which maximizes the product of the probability functions for all n voxels:

$$\boldsymbol{T}_k^{T2S} = \arg\max_{\boldsymbol{T}_k^{T2S}} \left(\prod_{i=1}^{n} p(\boldsymbol{T}_k^{T2S}, \boldsymbol{P}_{\text{source}}) \right) \tag{11}$$

Finally, the pose of the FV at the kth frame is determined using the following transformation:

$$\boldsymbol{E}_k^f = \left(\boldsymbol{T}_k^{T2S}\right)^{-1} * \boldsymbol{E}_o^l \tag{12}$$

Cross-vehicle registration considerations. The experiments conducted in CARLA mainly focused on representative leader–follower scenarios, and thus did not cover all possible cases where geometric overlap between the follower's current scan and the leader's historical map might become limited (e.g., large headways, lane changes, or temporary occlusions). In real-world deployment, however, robustness can be further enhanced by searching a bounded temporal window around the estimated index j, filtering candidate frames with a coarse overlap score before NDT refinement, and applying a robust outlier check. When the overlap is insufficient, the system can fall back to single-vehicle odometry and resume cross-vehicle corrections once sufficient overlap is restored. These mechanisms provide a practical pathway to extend our framework beyond the tested simulation cases.

D. IMU-ASSISTED LIGHTWEIGHT POINT CLOUD FEATURE EXTRACTION

In LF-CDA systems, the large size of LiDAR point clouds can impact real-time performance. Excessively large point clouds increase both the transmission time of the follower vehicle's point cloud and the time required for point cloud registration. To address this issue, rather than using raw point cloud data directly, we extract representative features. This approach not only significantly reduces data transmission volume but also retains sufficient information for effective point cloud registration and navigation processing.

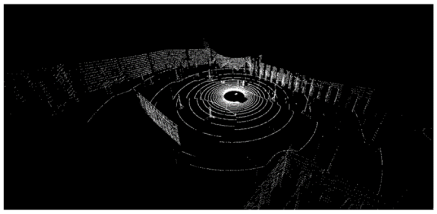
Common feature extraction methods include voxel grid filtering, key point extraction [51], and curvature-based sampling [46]. While key point extraction and curvature-based sampling capture rich local information, their high computational complexity can hinder real-time performance. To balance the richness of sampled information and computational efficiency, our feature extraction module primarily utilizes the voxel grid filtering method.

First, the original point cloud from the FV is processed using IMU data to remove distortions. Inspired by Qin et al. [52], the angular change between two consecutive IMU data points over a time interval is calculated as:

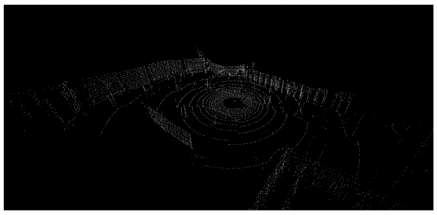
$$\boldsymbol{\alpha}_{n-1}^{n,IMU} = \frac{1}{2}(\boldsymbol{\omega}_{n-1} + \boldsymbol{\omega}_n) * dt \tag{13}$$

Considering the coupling of low-frequency LiDAR odometry with high-frequency IMU odometry, the angular change between two consecutive LiDAR frames can be determined by summing all $\boldsymbol{\alpha}_{n-1}^{n,IMU}$ values:

$$\boldsymbol{\alpha}_{k-1}^{k,Lidar} = \sum_{t_{k-1} \le t \le t_k} \boldsymbol{\alpha}_{n-1}^{n,IMU} \tag{14}$$



(a) Raw Point Cloud



(b) Feature Point Cloud

Fig. 4. Comparison between the raw point cloud and the feature point cloud after feature extraction.

The rotation matrix for the two consecutive LiDAR clouds $(k-1)^{\text{th}}$ and k^{th} frames can be estimated using the IMU sensor according to the following equation:

$$\boldsymbol{R}_{k-1}^{k,Lidar} = \boldsymbol{R}_{Lidar}^{IMU} * \boldsymbol{R}_{\boldsymbol{\alpha}_{k-1}^{k,Lidar}} * {\boldsymbol{R}_{Lidar}^{IMU}}^{-1} \tag{15}$$

For each point in k^{th} frame point cloud, its distortion can be removed using the equation:

$$\tilde{\boldsymbol{p}}_k^i = \frac{t_i}{t_{\text{scan}}} * \boldsymbol{R}_{k-1}^{k,Lidar} * \boldsymbol{p}_k^i \tag{16}$$

where t_i and t_{scan} represent the timestamp of the point within the kth frame and the scan time of the frame, respectively. For each point $\tilde{\boldsymbol{p}}_k^i = (x_i, y_i, z_i)$, the three-dimensional grid index (X, Y, Z) is calculated as follows:

$$X = \left\lfloor \frac{x_i}{\Delta} \right\rfloor, \quad Y = \left\lfloor \frac{y_i}{\Delta} \right\rfloor, \quad Z = \left\lfloor \frac{z_i}{\Delta} \right\rfloor \tag{17}$$

where Δ represents the side length of each voxel. For voxel (X, Y, Z), the arithmetic mean position of the points contained within it is calculated as:

$$\boldsymbol{p}_{\text{mid}} = \left(\frac{1}{N}\sum_{i=1}^{N} x_i, \frac{1}{N}\sum_{i=1}^{N} y_i, \frac{1}{N}\sum_{i=1}^{N} z_i \right) \tag{18}$$

where N is the number of points within the voxel and (x_i, y_i, z_i) are the coordinates of the ith point within the voxel. For each voxel, we select the point closest to the arithmetic mean position $\boldsymbol{p}_{\text{mid}}$ to represent the entire voxel, and remove the remaining points within the voxel, thereby updating the entire point cloud $\tilde{\boldsymbol{P}}_k$. This process ensures the integrity of the essential information in the point cloud while reducing the number of points.

Fig. 4 shows a comparison between a frame of the raw point cloud and the point cloud after feature extraction, clearly illustrating that feature extraction significantly reduces the number of points while effectively preserving environmental features.

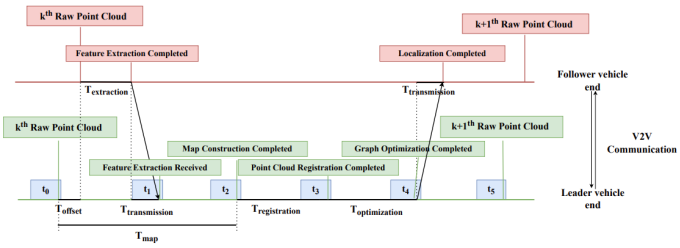


Fig. 5. Data flow and corresponding timeline.

E. DECOUPLED REAL-TIME SYNCHRONIZATION MODULE

From the moment the raw LiDAR point cloud is received, its flow through the system and the corresponding timeline are illustrated in Fig. 5. At the LV end, t_0, t_1, t_2, t_3, t_4, and t_5 represent the following events: t_0 marks the completion of the LV's raw point cloud reception, t_1 marks the completion of the FV's feature point cloud reception, t_2 represents the completion of the feature map update, t_3 is the completion of FV pose estimation, t_4 is the completion of global optimization, and t_5 represents the time when the next LV raw point cloud is received.

Additionally, T_{offset} denotes the synchronization offset between the LV and FV, $T_{\text{extraction}}$ indicates the time required for feature point extraction after the FV receives the raw point cloud, and $T_{\text{transmission}}$ represents the time taken for V2V information transmission between the LV and FV, which depends on the bandwidth capacity and the size of the data transmitted. T_{map} refers to the time taken by the LV to update the feature map using its feature point cloud and corresponding pose, $T_{\text{registration}}$ indicates the time consumed for point cloud registration, and $T_{\text{optimization}}$ represents the time spent on back-end graph optimization. To achieve real-time operation, the following constraints must be satisfied based on the timeline:

- The LV must receive the FV's feature points after the FV has completed feature point extraction, while also accounting for transmission time:

$$t_1 = t_0 + T_{\text{offset}} + T_{\text{extraction}} + T_{\text{transmission}} \tag{19}$$

- The feature map construction must be completed after the LV has received its raw point cloud and finished raw point cloud feature extraction:

$$t_2 = t_0 + T_{\text{extraction}} + T_{\text{map}} \tag{20}$$

- The completion of FV pose estimation must occur after both the feature map update and the reception of the FV's feature points:

$$t_3 = \max\{t_1, t_2\} + T_{\text{registration}} \tag{21}$$

- The back-end optimization module must be executed after the completion of FV pose estimation:

$$t_4 = t_3 + T_{\text{optimization}} \tag{22}$$

- For both the LV and FV, localization must be completed before the next frame of point cloud data is received:

$$t_4 \le t_5, \quad t_5 + T_{\text{transmission}} \le t_5 + T_{\text{offset}} \tag{23}$$

- For two consecutive point clouds:

$$t_5 = t_0 + H \tag{24}$$

where H is the frequency of the leader vehicle's LiDAR sensor.

Synchronization and communication considerations. In our CARLA experiments, the communication channel was assumed to be stable and did not fully capture latency jitter or clock misalignment. In real-world deployment, however, these issues can be addressed by maintaining a small index buffer around the nominal j, time-stamping feature packets and discarding stale candidates, and ensuring that the total processing time satisfies the timing constraints illustrated in Fig. 5. When the end-to-end delay exceeds a pre-specified bound, the system can simply skip cross-vehicle updates for that cycle to preserve real-time operation. These mechanisms provide a practical means to extend the proposed framework to more realistic communication conditions.

F. ITERATIVE NONLINEAR OPTIMIZATION SOLUTION ALGORITHM

In this subsection, we present an iterative nonlinear optimization algorithm to solve the cooperative localization problem formulated above via Equation (1)-(4). We adapted the solution framework from GTSAM [44], a C++ library specifically designed for solving large-scale optimization problems in robotics and computer vision, particularly those involving simultaneous localization and mapping. Below we detail each step of solving the problem, from initial estimates to iterative linearization and optimization, leading to convergence. The goal is to accurately estimate the poses of both the leader and follower vehicles by minimizing the total error across all measurements in the factor graph.

First, we re-format our objective function from Equation (1) as:

$$\min U(\boldsymbol{E}) = \sum_i \|\boldsymbol{e}_i(\boldsymbol{E})\|^2 \tag{25}$$

Here, $\boldsymbol{E}$ represents the errors for all poses involved in this graph optimization, $\boldsymbol{E} = \left(\boldsymbol{E}^l_{k-j}, \ldots, \boldsymbol{E}^l_k, \boldsymbol{E}^f_k\right)$. Next, Taylor expansion is used to approximate the error function as:

$$\boldsymbol{e}_i(\boldsymbol{E} + \Delta\boldsymbol{E}) \approx \boldsymbol{e}_i(\boldsymbol{E}) + \boldsymbol{J}_i \Delta\boldsymbol{E} \tag{26}$$

where $\Delta\boldsymbol{E}$ is the incremental update for the current poses, representing the latest LiDAR point clouds from both the leader and follower vehicles at the next frame k. Therefore, we have $\Delta\boldsymbol{E} = \left(\boldsymbol{E}^l_k, \boldsymbol{E}^f_k\right)$.

$\boldsymbol{J}_i$ is the Jacobian matrix with respect to the current estimate of poses $\boldsymbol{E}$:

$$\boldsymbol{J}_i = \frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}} = \begin{bmatrix} \frac{\partial \boldsymbol{e}_1}{\partial \boldsymbol{E}^l_{k-j}} & \cdots & \frac{\partial \boldsymbol{e}_1}{\partial \boldsymbol{E}^l_k} & \frac{\partial \boldsymbol{e}_1}{\partial \boldsymbol{E}^f_k} \\ \vdots & \ddots & \vdots & \vdots \\ \frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}^l_{k-j}} & \cdots & \frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}^l_k} & \frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}^f_k} \end{bmatrix} \tag{27}$$

where $\frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}^l_{k-j}}, \ldots, \frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}^l_k}$ are the partial derivatives of $\boldsymbol{e}_i$ with respect to the leader vehicle's poses at frames $k-j$ through

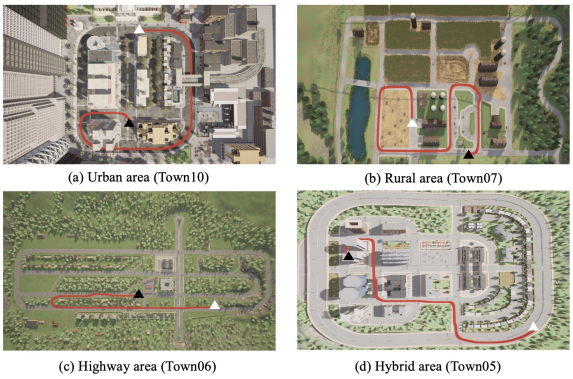


Fig. 6. The cruising path and environment, the starting point is marked with a black triangle, and the destination is marked with a white triangle.

k, and $\frac{\partial \boldsymbol{e}_i}{\partial \boldsymbol{E}_k^f}$ is the partial derivative of $\boldsymbol{e}_i$ with respect to the follower vehicle's poses at frame k.

By aggregating all linearized terms, we obtain a system of normal equations:

$$\boldsymbol{H}\Delta\boldsymbol{E} = -\boldsymbol{g} \tag{28}$$

where $\boldsymbol{H} = \sum_i \boldsymbol{J}_i^T \boldsymbol{J}_i$ is the Hessian matrix, representing the second-order partial derivatives of the error function, and $\boldsymbol{g} = \sum_i \boldsymbol{J}_i^T \boldsymbol{e}_i(\boldsymbol{E})$ is the gradient vector, indicating the direction in which each pose variable should be updated to reduce the error. Solving for $\Delta\boldsymbol{E}$ allows us to update the current estimates of all poses as follows:

$$\boldsymbol{E}_{\text{update}} = \boldsymbol{E} + \Delta\boldsymbol{E} \tag{29}$$

We repeat this process until $\Delta\boldsymbol{E}$ falls below a predefined threshold. These steps, implemented with the support of the GTSAM C++ library, enable the efficient and accurate solution of the graph optimization problem for cooperative localization in our LF-CDA system.

III. NUMERICAL EXPERIMENTS

To validate the performance of our proposed model, we conducted comprehensive experiments across various environments. Most commonly used datasets, such as Waymo Open, Apollo Scape, and KITTI, typically focus on the operation of a single vehicle. Conversely, datasets featuring Leader-Follower driving scenarios, like V2V4 Real, lack the IMU sensor data required for our model. Given that popular datasets are unsuitable for LF-CDA scenarios, we opted to construct various Leader-Follower scenarios within the CARLA simulation environment to evaluate the performance of our model.

To thoroughly assess the model's performance, we tested it in three different environments: urban, rural, highway and hybrid areas. We selected Town10, Town07, Town06, and Town05 in CARLA to represent these scenarios, respectively. Fig. 6 illustrates these maps along with the vehicle driving routes. During driving, we applied the vehicle-to-vehicle (V2V) communication range of 300 meters, as referenced in [53]

In this setup, a Velodyne 32-line LiDAR sensor was configured with the following parameters: 32 channels, with a rotation frequency of 20Hz. The sensor tick was set to 0.2 seconds, corresponding to a frequency of 5Hz. The IMU was configured to a frequency of 200Hz. For convenience, the LiDAR and IMU were coupled together. All experiments were conducted on a desktop computer equipped with an Intel Xeon W-2295 3.00 GHz CPU and 125 GB of RAM.

A. LOCALIZATION ACCURACY EVALUATION

To evaluate the performance of our proposed model, we used three benchmark models. The first is the CT-ICP algorithm [47], the second is the KISS-ICP algorithm [48], and the third one is the Traj-LO [54] algorithm, all of them representing state-of-the-art algorithms for localization tasks. The fourth method utilizes our proposed algorithm for cooperative localization of both vehicles but excludes the high-overlap view-based graph optimization module, allowing us to verify that this module effectively mitigates cumulative errors during system operation. All models are tested across urban, rural, and highway environments, using both leader and follower vehicle trajectories within the CARLA simulation. This benchmarking strategy is intended to ensure a comprehensive and fair assessment of localization accuracy and computational performance under conditions representative of real-world leader-follower cooperative driving automation.

For this research, the Evaluation of Odometry (EVO) serves as the primary tool for analyzing odometry and localization accuracy. EVO is a comprehensive framework designed to evaluate the performance of odometry algorithms by comparing estimated trajectories with ground truth data. It provides a detailed assessment of how well the estimated trajectories align with the ground truth. We utilize key metrics such as maximum error, mean error, and Root Mean Square Error (RMSE) of the absolute pose for our evaluation. The results are presented in Table III, and the results of the fifth experiment (the proposed model) are visualized in Fig. 7. It should be noted that, due to the asymmetric architecture design of LF-CDA systems, the follower vehicles are not equipped with high-performance computing capabilities or a full perception-localization-decision-making stack. Consequently, CT-ICP, KISS-ICP, or Traj-LO is not run on the follower vehicle, leaving the corresponding cells in Table III empty.

Across Table III, we observe the following:

- In all scenarios, the proposed model consistently exhibits the lowest mean and RMSE errors for both the leader vehicle and follower vehicle compared to the four benchmark algorithms, underscoring its crucial role in improving cooperative localization accuracy. For instance, in the urban scene, the proposed model achieves an RMSE of 0.599 for the LV and 0.631 for the FV, compared to 0.716, 0.616, 0.611 for the CT-ICP, KISS-ICP and Traj-Lo models, respectively.
- Across all environments, the proposed model performs best in urban settings, followed by rural and highway areas , lastly hybrids. This is likely due to the richer point cloud features available in urban environments. A similar trend is seen with the other four benchmark models,

TABLE III
RESULTS OF THE LOCALIZATION EXPERIMENTS

Scene	Method	Agent	Maximum Error	Mean Error	RMSE
Urban	CT-ICP	Leader	1.857	0.701	0.716
		Follower			
	Traj-LO	Leader	1.885	0.601	0.611
		Follower			
	KISS-ICP	Leader	1.793	0.621	0.616
		Follower			
	Benchmark 4	Leader	1.793	0.621	0.616
		Follower	2.013	0.685	0.681
	Ours	Leader	1.478	0.601	0.599
		Follower	1.524	0.641	0.631
Rural	CT-ICP	Leader	1.953	1.101	1.112
		Follower			
	Traj-LO	Leader	1.775	1.123	1.066
		Follower			
	KISS-ICP	Leader	1.901	1.032	1.035
		Follower			
	Benchmark 4	Leader	1.901	1.032	1.035
		Follower	2.257	1.098	1.099
	Ours	Leader	1.879	1.017	1.019
		Follower	2.202	1.044	1.042
Highway	CT-ICP	Leader	1.956	1.167	1.176
		Follower			
	Traj-LO	Leader	1.712	1.005	1.065
		Follower			
	KISS-ICP	Leader	1.912	1.095	1.083
		Follower			
	Benchmark 4	Leader	1.912	1.095	1.083
		Follower	2.231	1.172	1.166
	Ours	Leader	2.208	1.071	1.069
		Follower	2.222	1.124	1.112
Hybrid	CT-ICP	Leader	3.956	2.167	2.196
		Follower			
	Traj-LO	Leader	3.712	1.909	2.026
		Follower			
	KISS-ICP	Leader	3.902	1.918	2.041
		Follower			
	Benchmark 4	Leader	3.902	1.918	2.041
		Follower	5.324	2.972	3.126
	Ours	Leader	3.715	1.908	2.028
		Follower	5.878	2.918	3.118

where performance follows the order: urban $>$ rural $>$ highway $>$ hybrid. For example, in the urban scenario, the proposed model achieves an RMSE of 0.599 for the LV, whereas the RMSE increases to 1.019 in rural scenes, 1.069 on highways and 2.028 in hybrid. Notably, the LV generally exhibits higher localization accuracy compared to the FV across all tested models. This is likely because the pose of the follower vehicle is estimated based on the leader vehicle's pose. Consequently, any estimation error in the relative pose between the two vehicles results in greater accumulated error for the follower vehicle compared to the leader, which is a reasonable outcome.

- The benefits of the *asynchronous view repetition-based graph optimization model* are evident when comparing results with and without its use in our fourth benchmark. For example, in the urban scenario, the LV's localization RMSE improves from 0.616 without the graph optimization module to 0.599 with it. Similarly, in rural highway scenarios, the module reduces the LV's localization RMSE from 1.035 to 1.019, from 1.083 to 1.069, and from 2.041 to 2.028, respectively.
- An additional observation is that, across all baseline methods, the follower vehicle consistently exhibits larger localization errors than the leader vehicle. In contrast, the proposed framework substantially narrows this leader–follower performance gap, which directly demonstrates its suitability for asymmetric LF–CDA systems.

In words, the proposed model consistently achieves the lowest mean and RMSE errors for both the leader and follower vehicles across all environments, outperforming the baseline SLAM algorithms. This highlights the advantage of leveraging asynchronous view repetition and cooperative graph optimization in the leader-follower scenario. The improved accuracy of our approach is particularly beneficial for deployment in GNSS-denied environments, where reliable localization is critical for safe and coordinated vehicle operation. Fig. 7 further illustrates the superior performance of the model, with the localized trajectories closely aligning with the ground truth trajectories.

B. COMPUTATIONAL PERFORMANCE EVALUATION

To ensure that the proposed model operates in real-time, the entire system's cooperative localization process must be completed before the next LiDAR point cloud frame is received. As specified in the system setup, the sensor tick is set to 0.2 seconds, corresponding to a frequency of 5 Hz. This means that, for real-time performance, data processing must be completed within 200 ms, before the arrival of the next LiDAR frame. Consistent with the above analysis, we use CT-ICP, KISS-ICP, and Traj-LO as benchmarks. Since the follower vehicle lacks sufficient computational resources to independently run these algorithms, we adopt a setup where the follower transmits its raw point cloud data to the leader vehicle. At the leader's end, the algorithm will run separately to register the point clouds for both the leader and follower vehicles. Additionally, we analyze the performance of the proposed model without graph optimization or feature extraction to understand the computational burden of these modules individually. We aim to test whether the data transmission order and timing comply with the constraints of this module to verify that the system can operate in real time.

To evaluate the real-time capability of our proposed model, we measured the time consumed by each module and calculated the total time required for cooperative localization in the urban scenario (CARLA Town 10). A V2V communication channel, as described in [55], with a transmission speed of 27 Mb/s, was used. Initially, the raw point cloud ply file data is approximately 700 kb per frame(on average). After the feature extraction process, the file size is reduced to around 350 KB per frame. The results of this experiment are presented in Table IV, where the values represent the average computational cost of the module during vehicle operation. Notably, in our proposed model, the point registration time specifically refers to the feature map-based follower vehicle pose estimation module, which in our experiments operates in a multi-threaded manner, allowing multiple point registrations to be completed simultaneously, while for the CT-ICP, KISS-ICP and Traj-LO algorithms, it indicates the time required to complete pose estimation for both the leader and follower vehicles.

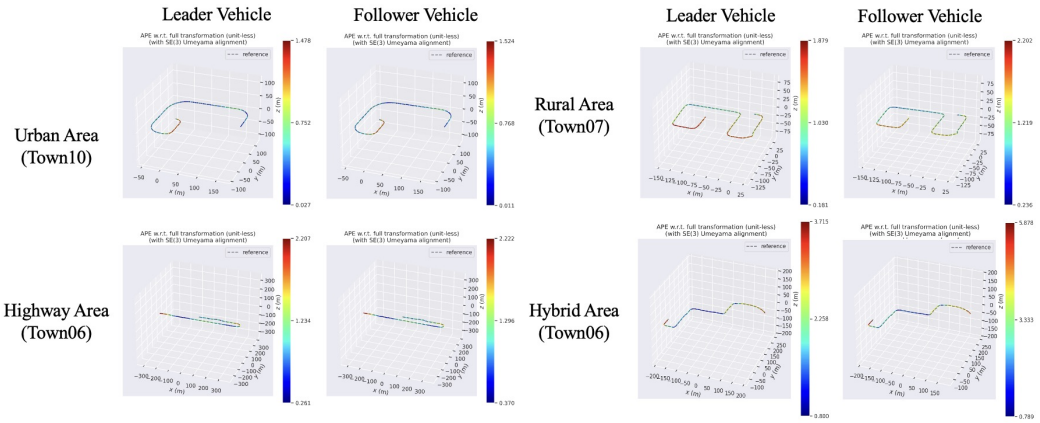


Fig. 7. Visualization of cooperative localization results of the proposed method in three different scenarios.

TABLE IV
RESULTS OF THE REAL-TIME PERFORMANCE ASSESSMENT

Method	**Feat. Extr.**	**V2V Trans.**	**Map Update**	**Point Reg.**	**Graph Opt.**	**Total**
Proposed Model	12.3ms	12.7ms	70.2ms	73.4ms	11.3ms	179.9ms
Proposed Model (No Graph Opt.)	12.3ms	12.7ms	70.2ms	73.4ms	0ms	168.6ms
Proposed Model (No Feat. Extr.)	0ms	25.4ms	75.9ms	207.7ms	12.3ms	321.3ms
CT-ICP		25.4ms		151.3ms		176.7ms
KISS-ICP		25.4ms		138.3ms		163.7ms
Traj-LO		25.4ms		201.4ms		226.8ms

From Table IV , we observe the following:

- The proposed model completes cooperative localization in 179.9ms, demonstrating satisfactory runtime performance. The most time-consuming component is the point registration, taking 73.4ms, followed by the feature map update, which takes 70.2ms.
- Compared to the three state-of-the-art benchmark algorithms, *CT-ICP*, *KISS-ICP*, and *Traj-LO* which take 176.7ms, 163.7ms and 226.8ms respectively for LF-CDA localization, the proposed algorithm enables real-time operation with performance comparable to or even slightly better than these leading algorithms.
- Removing the *asynchronous view-based graph optimization module* reduces processing time to 168.6ms, further enhancing computational efficiency. However, as discussed previously, this also results in a significant decrease in modeling accuracy, indicating a trade-off between computational speed and accuracy.
- In contrast, removing the *IMU-assisted lightweight point cloud feature extraction module* increases processing time substantially to 321.3ms, highlighting the essential role of IMU-assisted feature extraction in ensuring real-time performance.
- Beyond the overall reduction in runtime, the proposed framework also achieves a more balanced distribution of processing time across modules. In contrast to baselines such as CT-ICP and Traj-LO, where point registration dominates the runtime, our design prevents single-module bottlenecks, leading to more predictable and stable real-time performance in diverse environments.

Table IV demonstrates that our model not only delivers high localization accuracy but also maintains real-time performance, with a total computation time per frame well below the LiDAR frame period. This ensures that the proposed system is suitable for real-world deployment in leader-follower scenarios, where both accuracy and computational efficiency are essential for stable and responsive operation.

C. SENSITIVITY ANALYSIS

In this section, we performed a sensitivity analysis using the proposed model to examine the impact of different operating environments (Highway, Urban, and Rural areas) and LiDAR point cloud sizes on system performance. The goal of this analysis is to understand how the system's stability and accuracy vary under different conditions. Specifically, we evaluated three key metrics: runtime, RMSE, and Mean Error. These metrics provide a comprehensive view of the system's performance across various point cloud sizes and environmental conditions, helping us identify strengths and areas for improvement.

This sensitivity analysis aims to provide data support for future enhancements and optimizations, ensuring the system can operate stably and efficiently in diverse real-world scenarios. In the analysis, aside from the LiDAR point cloud size, all other features remain unchanged. The results are shown in Table V, where the "Point Cloud Size" refers to the LiDAR's capability, specifically the number of points the LiDAR can emit per second. The "Average Raw Point Cloud" represents the average number of points received by the LiDAR per frame under this capability. The "Average Mean Error" and "Average

RMSE" are the mean values for the LV and FV, measuring the localization performance of the model.

TABLE V
RESULTS OF THE SENSITIVITY ANALYSIS

Point Cloud Size /Second	Scenario	Average Raw Point Cloud /Frame	Runtime /ms	Average Mean Error	Average RMSE
720,000	Urban	16530	156.4	0.722	0.712
	Rural	14290	151.6	1.185	1.199
	Highway	10330	142.8	1.201	1.193
	Hybrid	15390	155.8	2.601	2.653
960,000	Urban	22640	165.6	0.652	0.651
	Rural	19005	160.6	1.105	1.099
	Highway	13050	149.9	1.141	1.143
	Hybrid	20440	165.8	2.602	2.633
1,200,000	Urban	27580	179.9	0.621	0.615
	Rural	23890	169.6	1.031	1.031
	Highway	16550	156.6	1.098	1.091
	Hybrid	25550	171.8	2.413	2.573
1,440,000	Urban	33040	200.2		
	Rural	28990	180.5	0.992	0.971
	Highway	19880	161.7	1.120	1.116
	Hybrid	30240	191.8	2.432	2.571
1,680,000	Urban	37880	205.7		
	Rural	33870	200.9		
	Highway	23450	167.7	1.101	1.096
	Hybrid	34670	203.6		

As shown in Table V:

- As the point cloud size increases, the system's runtime also rises due to the additional computational load required to process larger datasets. However, the system continues to demonstrate a satisfactory runtime lower than 200ms, meeting the computational requirements effectively.
- Despite slightly longer runtimes, the average mean error and RMSE generally decrease, indicating improved localization accuracy. For example, in the urban scenario, the average mean error decreases from 0.722 at 720,000 points/second to 0.621 at 1,200,000 points/second, with a similar trend in RMSE. These results demonstrate the trade-off between computational efficiency and accuracy, emphasizing the need for optimization to maintain real-time performance while ensuring high localization precision.
- Localization accuracy varies across different scenarios, even when point cloud sizes are similar. For instance, with a LiDAR point cloud density of 720,000 points per second in an urban environment, an average of 16,530 points per frame yields a mean localization error of 0.722. In contrast, at a LiDAR point cloud density of 1,200,000 points per second on a highway, where the average points per frame is slightly higher at 16,550, the mean localization error rises significantly to 1.098. This suggests that the system's accuracy is not solely dependent on point cloud size. A possible explanation is that rural or highway environments have fewer distinctive point cloud features, making point cloud registration more challenging and leading to reduced localization accuracy.
- Although the hybrid scenarios have a comparable or even larger number of LiDAR points per frame, their localization accuracy is significantly lower. This is mainly because hybrid environments combine various road types and scene transitions, leading to inconsistent geometric features and sudden environmental changes. These factors make point cloud registration and pose estimation more challenging, resulting in higher localization errors compared to single-type scenarios.

The sensitivity analysis in Table V shows that as the point cloud size increases, localization accuracy improves while the runtime remains within practical limits. This trade-off supports flexible system tuning for various deployment scenarios, ensuring that the approach remains robust and scalable for different hardware capabilities and environmental conditions.

D. Applicability and Limitations

The comparative results demonstrate that the proposed asynchronous view repetition-based cooperative localization framework achieves significant improvements in both localization accuracy and computational efficiency compared to leading LiDAR-based SLAM baselines. These advances directly address the practical requirements of asymmetric leader–follower systems, where follower vehicles operate under stringent computational and communication constraints. By leveraging the unique characteristics of the leader–follower paradigm and optimizing multi-vehicle pose estimation through graph-based methods, our approach ensures robust and scalable localization even when GNSS signals are unavailable or degraded. This makes the proposed system particularly suitable for real-world deployment in applications such as convoy driving, platooning, and emergency response operations, where reliable, real-time, and resource-efficient localization is essential.

Nevertheless, the framework has two main conditions that define its applicability. First, in rare situations where an object is static when scanned by both vehicles but moves significantly in the interval between scans, the algorithm may incorrectly associate the object as a stable feature. This can reduce localization accuracy, but such cases are uncommon in practice, since the time interval and inter-vehicle distance in a leader–follower setting are typically moderate by design. Thus, while the method is generally robust to dynamic environments, extreme cases involving rapid object motion exactly between scans remain challenging. Second, the effectiveness of the view-repetition approach relies on the recurrence of similar viewpoints to maintain consistent geometric context. In long, non-repetitive routes—such as point-to-point deliveries or exploratory mapping in unfamiliar areas—the opportunity for overlap is limited, which reduces the strength of cross-vehicle constraints. In such scenarios, the system necessarily falls back on single-frame odometry, and robustness decreases compared to cases with frequent viewpoint repetition. This limitation can be alleviated by hybridizing with complementary sensors (e.g., visual–inertial odometry) or by adjusting the reliance on historical frames based on online similarity checks.

Importantly, both of these challenges are well-understood and can be effectively managed in practice through design constraints, fallback strategies, and integration with complementary sensing modalities. As such, they do not undermine

the overall applicability of the framework, but rather clarify the boundary conditions under which it is expected to deliver reliable performance.

IV. CONCLUSION

In this research, we introduced a lightweight LiDAR-based cooperative localization model for asymmetric LF-CDA, designed to provide stable localization in GNSS-compromised environments. The proposed model uniquely leverages the key characteristics of the LF-CDA system, termed "asynchronous view repetition", and formulates an asynchronous view repetition-based graph optimization model to minimize positional errors for both leader and follower vehicles. To provide input to and solve the graph optimization model, a lightweight cooperative localization framework with multiple submodules is established, allowing the system to function independently of environmental constraints.

We implemented the developed modeling framework and demonstrated its compatibility with the asymmetric architecture of LF-CDA systems. To validate its performance, we conducted comprehensive experiments across various environments, including urban, rural, and highway areas. We evaluated the model against three benchmarks: the CT-ICP algorithm, the KISS-ICP algorithm, and our proposed algorithm without the graph optimization module. In all scenarios, the proposed model consistently exhibited the lowest mean and RMSE errors for both vehicles, highlighting its critical role in improving cooperative localization accuracy. We also tested the computational efficiency of the proposed algorithm, which completed cooperative localization in 179.9ms per frame, meeting real-time performance requirements and maintaining comparable or even superior computational performance. Sensitivity analysis showed that as point cloud size increases, the system's runtime rises due to the added computational load, but remains acceptable. Despite longer runtimes, the average mean error and RMSE generally decrease, indicating improved localization accuracy.

Looking forward, while this work focuses on a LiDAR-based framework due to the robustness, scale consistency, and computational efficiency LiDAR offers for resource-constrained follower vehicles, we recognize the rapid advances in multi-sensor fusion for cooperative localization. Integrating complementary modalities—such as cameras, radar, and IMUs—has the potential to further enhance accuracy and reliability, particularly in environments where any single sensor may encounter limitations. Future research will explore how to efficiently incorporate visual and inertial cues, and how to design lightweight, tightly-coupled fusion algorithms that remain practical for real-time deployment in asymmetric LF-CDA systems as hardware capabilities continue to evolve.

REFERENCES

[1] D. Juling, "Truck platooning: Great prospect or unrealistic concept for military logistics in europe?" *The RUSI Journal*, vol. 168, no. 1-2, pp. 108–115, 2023.

[2] S. Van De Hoef, K. H. Johansson, and D. V. Dimarogonas, "Fuel-efficient en route formation of truck platoons," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 102–112, 2017.

[3] V. Turri, B. Besselink, and K. H. Johansson, "Cooperative look-ahead control for fuel-efficient and safe heavy-duty vehicle platooning," *IEEE Transactions on Control Systems Technology*, vol. 25, no. 1, pp. 12–28, 2016.

[4] C. E. M. Johnson, C. E. D. Rothblatt, and D. C. Overton, "The future is now at JRTC: Modernizing the force with tactical wheeled vehicle leader–follower technology," [Online]. Available: https://www.army.mil/article/249257/the_future_is_now_at_jrtc_modernizing_the_force_with_tactical_wheeled_vehicle_leader_follower_technology, 2021, accessed: 2025-09-14.

[5] Q. Tang, Y. Cheng, X. Hu, C. Chen, Y. Song, and R. Qin, "Evaluation methodology of leader-follower autonomous vehicle system for work zone maintenance," *Transportation research record*, vol. 2675, no. 5, pp. 107–119, 2021.

[6] Q. Tang, X. Hu, and R. Qin, "Development of operation guidelines for leader–follower autonomous maintenance vehicles at work zone locations," *Transportation Research Record*, vol. 2676, no. 3, pp. 696–711, 2022.

[7] S.-L. Dai, S. He, H. Cai, and C. Yang, "Adaptive leader–follower formation control of underactuated surface vehicles with guaranteed performance," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 3, pp. 1997–2008, 2020.

[8] G. L. Mariottini, G. Pappas, D. Prattichizzo, and K. Daniilidis, "Vision-based localization of leader-follower formations," in *Proceedings of the 44th IEEE Conference on Decision and Control*. IEEE, 2005, pp. 635–640.

[9] J. Huang, S. M. Farritor, A. Qadi, and S. Goddard, "Localization and follow-the-leader control of a heterogeneous group of mobile robots," *IEEE/ASME transactions on Mechatronics*, vol. 11, no. 2, pp. 205–215, 2006.

[10] S. Saeedi, M. Trentini, M. Seto, and H. Li, "Multiple-robot simultaneous localization and mapping: A review," *Journal of Field Robotics*, vol. 33, no. 1, pp. 3–46, 2016.

[11] M. J. Jiménez-Martínez, M. Farjas-Abadia, and N. Quesada-Olmo, "An approach to improving gnss positioning accuracy using several gnss devices," *Remote Sensing*, vol. 13, no. 6, p. 1149, 2021.

[12] E. D. Kaplan and C. Hegarty, *Understanding GPS/GNSS: Principles and Applications*. Norwood, MA, USA: Artech House, 2017.

[13] S. Liu, Z. Yao, X. Cao, X. Cai, and M. Sheng, "Enhancing navigation adaptability for ground vehicles: Fine-grained urban environment recognition based on gnss measurements," *IEEE Transactions on Instrumentation and Measurement*, 2024.

[14] A. Angrisano, C. Gioia, S. Gaglione, and G. Del Core, "Gnss reliability testing in signal-degraded scenario," *International Journal of Navigation and Observation*, vol. 2013, no. 1, p. 870365, 2013.

[15] A. Innac, M. Z. H. Bhuiyan, S. Söderholm, H. Kuusniemi, and S. Gaglione, "Reliability testing for multiple gnss measurement outlier detection," in *2016 European Navigation Conference (ENC)*. IEEE, 2016, pp. 1–8.

[16] Lynn, "Imu, gnss, and slam in mobile mapping systems and beyond," 2023, available from: https://www.mosaic51.com/technology/imu-gnss-slam-technology-mobile-mapping.

[17] X. Shan, A. Cabani, and H. Chafouk, "Cooperative vehicle localization in multi-sensor multi-vehicle systems based on an interval split covariance intersection filter with fault detection and exclusion," *Vehicles*, vol. 6, no. 1, pp. 352–373, 2024.

[18] J. J. Morales, P. F. Roysdon, and Z. M. Kassas, "Signals of opportunity aided inertial navigation," in *Proc. ION GNSS+*, 2016, pp. 1492–1501.

[19] K. Shamaei, J. Khalife, and Z. M. Kassas, "Performance characterization of positioning in LTE systems," in *Proc. ION GNSS+*, 2016, pp. 2262–2270.

[20] D. Gulati, F. Zhang, D. Clarke, and A. Knoll, "Vehicle infrastructure cooperative localization using factor graphs," in *2016 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2016, pp. 1085–1090.

[21] M.-j. Kim, O. Kwon, and J. Kim, "Vehicle to infrastructure-based lidar localization method for autonomous vehicles," *Electronics*, vol. 12, no. 12, p. 2684, 2023.

[22] A. Jamshidpey, M. Wahby, M. K. Heinrich, M. Allwright, W. Zhu, and M. Dorigo, "Centralization vs. decentralization in multi-robot coverage: Ground robots under uav supervision," *arXiv preprint arXiv:2408.06553*, 2024.

[23] Z. Jin and C. Jiang, "Range-aided lidar-inertial multi-vehicle localization and mapping in degenerate environments," *IEEE Transactions on Intelligent Vehicles*, 2024.

[24] K. Nagatani, Y. Okada, N. Tokunaga, S. Kiribayashi, K. Yoshida, K. Ohno, E. Takeuchi, S. Tadokoro, H. Akiyama, I. Noda *et al.*, "Multirobot exploration for search and rescue missions: A report on

map building in robocuprescue 2009," *Journal of Field Robotics*, vol. 28, no. 3, pp. 373–387, 2011.
[25] N. Michael, S. Shen, K. Mohta, V. Kumar, K. Nagatani, Y. Okada, S. Kiribayashi, K. Otake, K. Yoshida, K. Ohno *et al.*, "Collaborative mapping of an earthquake damaged building via ground and aerial robots," in *Field and service robotics: results of the 8th international conference*. Springer, 2014, pp. 33–47.
[26] L. Riazuelo, J. Civera, and J. M. Montiel, "C2tam: A cloud framework for cooperative tracking and mapping," *Robotics and Autonomous Systems*, vol. 62, no. 4, pp. 401–413, 2014.
[27] R. Dubé, A. Gawel, H. Sommer, J. Nieto, R. Siegwart, and C. Cadena, "An online multi-robot slam system for 3d lidars," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 1004–1011.
[28] S. Fang, H. Li, and M. Yang, "Adaptive cubature split covariance intersection filter for multi-vehicle cooperative localization," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1158–1165, 2021.
[29] T. Cieslewski, S. Choudhary, and D. Scaramuzza, "Data-efficient decentralized visual slam," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 2466–2473.
[30] X. S. Zhou and S. I. Roumeliotis, "Multi-robot slam with unknown initial correspondence: The robot rendezvous case," in *2006 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2006, pp. 1785–1792.
[31] L. Paull, G. Huang, M. Seto, and J. J. Leonard, "Communication-constrained multi-auv cooperative slam," in *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2015, pp. 509–516.
[32] A. Napier, G. Sibley, and P. Newman, "Real-time bounded-error pose estimation for road vehicles using vision," in *13th International IEEE Conference on Intelligent Transportation Systems*. IEEE, 2010, pp. 1141–1146.
[33] B. Kim, M. Kaess, L. Fletcher, J. Leonard, A. Bachrach, N. Roy, and S. Teller, "Multiple relative pose graphs for robust cooperative mapping," in *2010 IEEE International Conference on Robotics and Automation*. IEEE, 2010, pp. 3185–3192.
[34] R. Aragüés, E. Montijano, and C. Sagüés, "Consistent data association in multi-robot systems with limited communications," 2011.
[35] D. Tardioli, E. Montijano, and A. R. Mosteo, "Visual data association in narrow-bandwidth networks," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2015, pp. 2572–2577.
[36] C. Forster, S. Lynen, L. Kneip, and D. Scaramuzza, "Collaborative monocular slam with multiple micro aerial vehicles," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2013, pp. 3962–3970.
[37] D. Zou and P. Tan, "Coslam: Collaborative visual slam in dynamic environments," *IEEE transactions on pattern analysis and machine intelligence*, vol. 35, no. 2, pp. 354–366, 2012.
[38] A. Cunningham, M. Paluri, and F. Dellaert, "Ddf-sam: Fully distributed slam using constrained factor graphs," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3025–3030.
[39] A. Cunningham, V. Indelman, and F. Dellaert, "Ddf-sam 2.0: Consistent distributed smoothing and mapping," in *2013 IEEE international conference on robotics and automation*. IEEE, 2013, pp. 5220–5227.
[40] X. Wu, J. Xu, P. Hu, G. Wang, and H. Wang, "Lhmap-loc: Cross-modal monocular localization using lidar point cloud heat map," *arXiv preprint arXiv:2403.05002*, 2024.
[41] S. Lynen, T. Sattler, M. Bosse, J. A. Hesch, M. Pollefeys, and R. Siegwart, "Get out of my lab: Large-scale, real-time visual-inertial localization," in *Proc. Robotics: Science and Systems (RSS)*, 2015.
[42] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g2o: A general framework for graph optimization," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 3607–3613.
[43] S. Agarwal, K. Mierle *et al.*, "Ceres solver: Tutorial & reference," *Google Inc*, vol. 2, no. 72, p. 8, 2012.
[44] F. Dellaert, "Factor graphs and gtsam: A hands-on introduction," Georgia Institute of Technology, Tech. Rep. GT-RIM-CP&R-2012-002, 2012.
[45] D. M. Rosen, L. Carlone, A. S. Bandeira, and J. J. Leonard, "Se-sync: A certifiably correct algorithm for synchronization over the special euclidean group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019.
[46] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time," in *Proc. Robotics: Science and Systems (RSS)*, Berkeley, CA, USA, 2014.
[47] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, "Ct-icp: Real-time elastic lidar odometry with loop closure," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 5580–5586.
[48] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "Kiss-icp: In defense of point-to-point icp–simple, accurate, and robust registration if done the right way," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 1029–1036, 2023.
[49] D. Chetverikov, D. Svirko, D. Stepanov, and P. Krsek, "The trimmed iterative closest point algorithm," in *2002 International Conference on Pattern Recognition*, vol. 3. IEEE, 2002, pp. 545–548.
[50] M. Magnusson, "The three-dimensional normal-distributions transform: An efficient representation for registration, surface analysis, and loop detection," Ph.D. dissertation, Örebro University, Örebro, Sweden, 2009.
[51] B. Steder, R. B. Rusu, K. Konolige, and W. Burgard, "Narf: 3d range image features for object recognition," in *Proc. IROS Workshop on Defining and Solving Realistic Perception Problems in Personal Robotics*, 2010.
[52] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE transactions on robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
[53] R. Xu, H. Xiang, X. Xia, X. Han, J. Li, and J. Ma, "Opv2v: An open benchmark dataset and fusion pipeline for perception with vehicle-to-vehicle communication," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 2583–2589.
[54] X. Zheng and J. Zhu, "Traj-lo: In defense of lidar-only odometry using an effective continuous-time trajectory," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1961–1968, 2024.
[55] L. Gao, H. Xiang, X. Xia, and J. Ma, "Multi-sensor fusion for vehicle-to-vehicle cooperative localization with object detection and point cloud matching," *IEEE Sensors Journal*, 2024.



Yuxin Ding received his Bachelor degree from the Civil Aviation University Of China, and Master degree from Georgia Institute Of Technology. He is currently pursuing the Ph.D. with the Department of Civil and Environmental Engineering, Penn State University, Pennsylvania, USA. His research is focused on autonomous vehicles and dynamic system modeling.



Chenxi Chen received the B.Eng. degree from the Department of Civil and Environmental Engineering, Zhejiang University, China, the M.Eng. degree from Missouri University of Science and Technology, USA, and the Ph.D. degree from the Department of Civil and Environmental Engineering, Penn State University, USA. His research is focused on autonomous vehicles, machine learning, and mobility data analysis.



Xianbiao Hu is currently an Associate Professor with the Pennsylvania State University, in its Civil and Environmental Engineering Department. He received the Ph. D. degree from the University of Arizona, Tucson, AZ in 2013. His current research is focus on smart mobility systems, connected and automated vehicles, electric vehicles, mobility behavior management, transportation big data analytics, and traffic flow and system modeling. He is an Associate Editor of IEEE Transactions on Intelligent Transportation Systems.